



**Московский государственный технический университет
Факультет ИУ «Информатика и системы управления»
Кафедра ИУ-1 «Системы автоматического управления»**

ОТЧЕТ
по лабораторной работе №2
по дисциплине
«Эргатические системы»

Выполнил: Чжу Цзыюй

Группа: ИУ1И-41М

Проверил: Андриков Д. А.

Работа выполнена: 24/03/2026

Отчет сдан: 24/03/2026

Оценка:

1. Цель работы

- Освоить методы предварительной обработки и визуализации кардиологических данных с помощью **PCA** и **t-SNE**.
- Исследовать современные фреймворки автоматического машинного обучения (AutoML), обосновать выбор одного из них.
- Применить выбранный AutoML-инструмент для построения модели бинарной классификации состояния здоровья (`Healthy_Status`) на основе числовых параметров ЭКГ.
- Оценить качество модели с помощью **матрицы ошибок (confusion matrix)** и метрики **F1**, а также других стандартных показателей классификации.

2. Исходные данные

Источник:

Датасет предоставлен преподавателем и доступен по ссылке:

<https://raw.githubusercontent.com/AI-is-out-there/data2lab/refs/heads/main/модуль%203%20-%20датасет%20-%20практика.csv>

В соответствии с заданием загружены первые **5000 строк**.

После отбора релевантных столбцов получена таблица со следующими признаками:

Признак	Описание
<code>Count_subj</code>	Номер обследуемого
<code>rr_interval</code>	Длительность RR-интервала (мс)
<code>p_end</code>	Время окончания зубца Р (отсчёты)
<code>qrs_onset</code>	Время начала комплекса QRS (отсчёты)
<code>qrs_end</code>	Время окончания комплекса QRS (отсчёты)
<code>p_axis</code>	Электрическая ось зубца Р (градусы)
<code>qrs_axis</code>	Электрическая ось комплекса QRS (градусы)
<code>t_axis</code>	Электрическая ось зубца Т (градусы)
<code>Healthy_Status</code>	Целевая переменная 0 – пациент нездоров 1 – пациент здоров

Статистика классов после удаления пропусков:

Класс	Количество	Доля
Нездоров (0)	4040	80.8%
Здоров (1)	960	19.2%
Всего	5000	100%

Важно: наблюдается значительный дисбаланс классов – здоровых пациентов примерно в 4.2 раза меньше, чем нездоровых.

3. Предварительная обработка и разведочный анализ (EDA)

3.1. Очистка данных

Пропущенные значения в выбранных столбцах отсутствовали.

Целевая переменная приведена к целочисленному типу `int`.

Выбросы (например, `p_end = 29999`) были сохранены для чистоты эксперимента.

3.2. Формирование обучающей и тестовой выборок

Данные разделены в соотношении 80/20 с сохранением стратификации по классу:

- **Обучающая выборка:** 4000 объектов
- **Тестовая выборка:** 1000 объектов

3.3. Стандартизация признаков

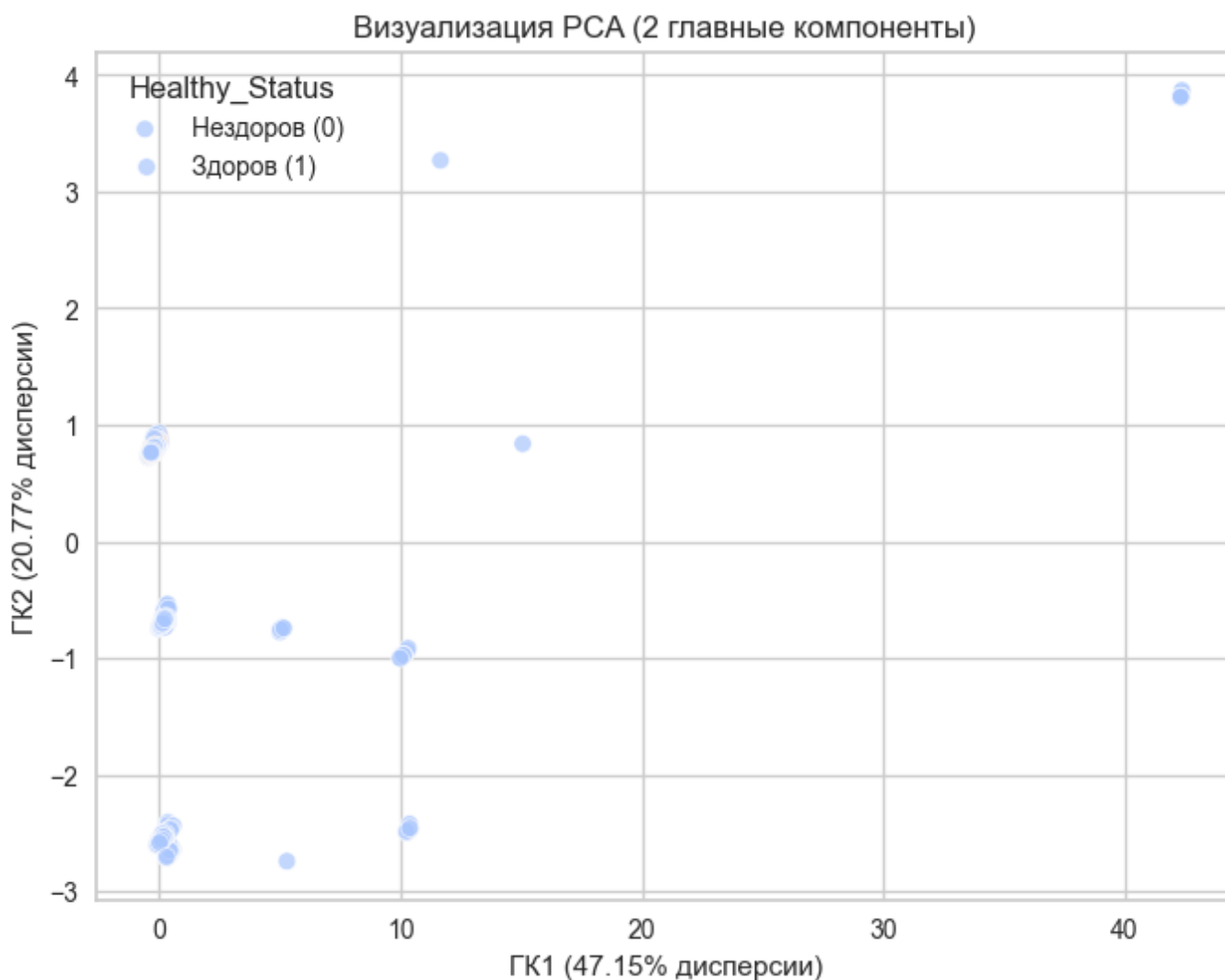
Применён `StandardScaler` для приведения всех числовых признаков к нулевому среднему и единичному стандартному отклонению.

3.4. Уменьшение размерности и визуализация

РСА (главных компонент – 2)

Две первые главные компоненты объясняют:

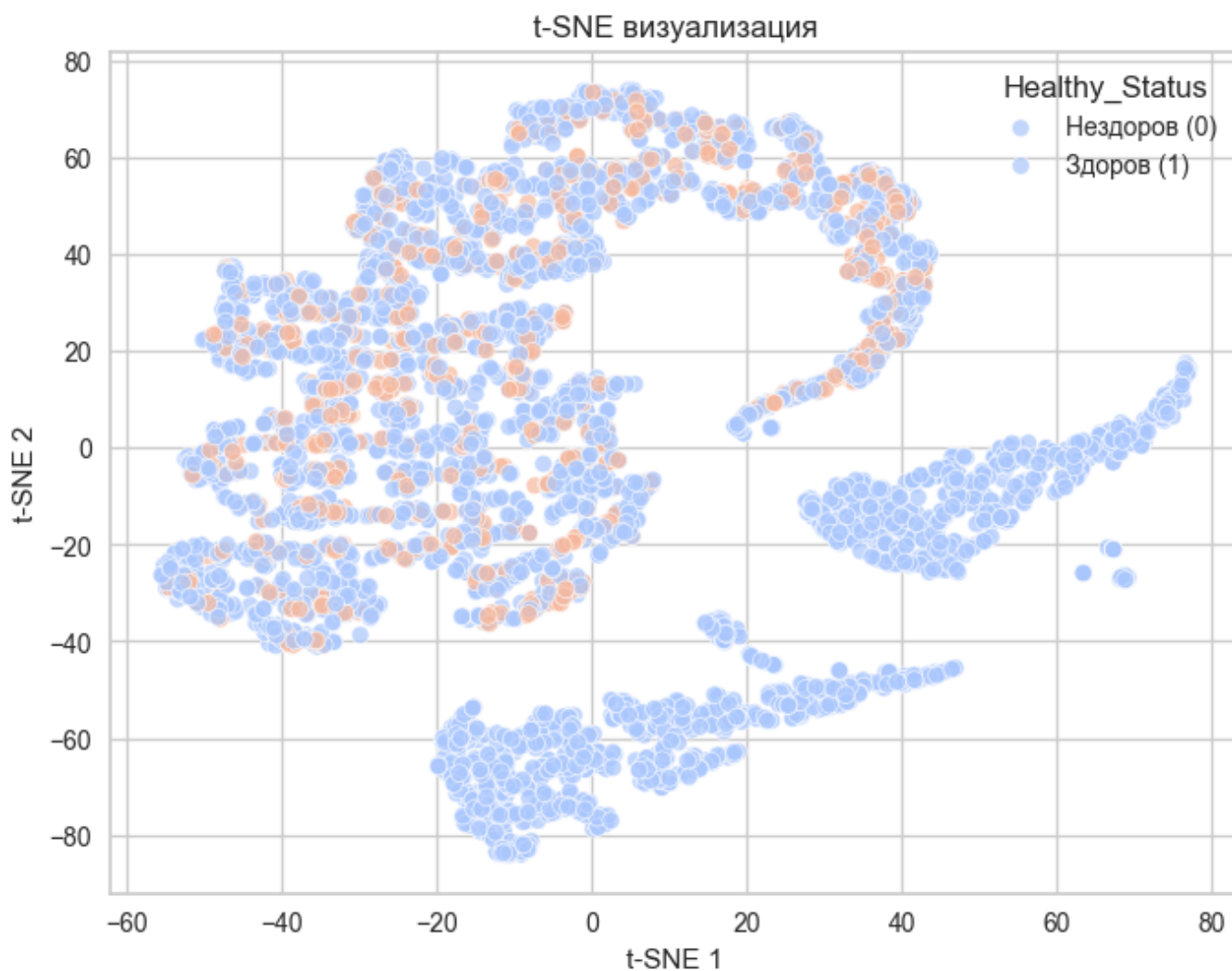
- ГК1: **47.15%** дисперсии
 - ГК2: **20.77%** дисперсии
- Суммарно $\approx 67.9\%$ информации.**



Классы частично перекрываются, но прослеживается некоторая кластеризация: класс `0` (нездоров) сконцентрирован слева, а класс `1` (здоров) имеет более рассеянную структуру.

t-SNE (число компонент – 2, perplexity=30)

t-SNE лучше разделяет локальные кластеры, формируя несколько изолированных скоплений здоровых пациентов.



Вывод: данные содержат нелинейные зависимости, поэтому классификатор должен уметь улавливать сложные закономерности.

4. Выбор AutoML-фреймворка

Рассмотрены следующие инструменты:

Фреймворк	Преимущества	Недостатки
AutoGluon	Высокая точность «из коробки»; поддержка текста и изображений	Громоздкие зависимости, сложнее в настройке
H2O AutoML	Мощный, множество алгоритмов; Java-бэкенд	Требует запуска H2O-кластера; менее pythonic API
TPOT	Генетический поиск пайплайнов; интерпретируемость	Медленная работа; ограниченный набор моделей
PyCaret	Минимум кода; встроенная визуализация; быстрое сравнение >20 моделей	Меньшая гибкость в нестандартных конвейерах

Обоснование выбора PyCaret:

- Удобство для учебного процесса** – все этапы (`setup` , `compare_models` , `tune_model` , `predict_model`) реализуются в несколько строк.
- Автоматическая предобработка** – заполнение пропусков, масштабирование, балансировка классов.
- Встроенная визуализация** – матрица ошибок, ROC-кривая, важность признаков.
- Прозрачное сравнение моделей** – таблица с метриками сразу после вызова `compare_models()` .
- Стабильность и активное развитие.**

Таким образом, для данной лабораторной работы выбран **PyCaret (модуль классификации)**.

5. Построение модели и настройка

5.1. Инициализация PyCaret

Среда сконфигурирована со следующими параметрами:

- `normalize=True`
- `fix_imbalance=True` с использованием **SMOTE(sampling_strategy=0.4)** – миноритарный класс дополняется синтетическими примерами до достижения 40% от размера мажоритарного класса.
- `session_id=42` – для воспроизводимости.
- `train_size=0.8` – внутри кросс-валидации 80% данных используется для обучения.

5.2. Сравнение моделей

Вызвана `compare_models(sort='F1', n_select=3)` . Обучены и провалидированы (10-кратная кросс-валидация) следующие алгоритмы:

AdaBoost, CatBoost, Gradient Boosting, Random Forest, LightGBM, Extra Trees, XGBoost, Decision Tree, SVM, QDA, Logistic Regression, KNN, Naive Bayes, LDA, Ridge, Dummy.

Топ-3 по F1 (среднее по фолдам):

Модель	Accuracy	AUC	Recall	Prec.	F1
Gradient Boosting Classifier	0.854	0.912	0.764	0.594	0.668
Ada Boost Classifier	0.844	0.908	0.783	0.568	0.658
CatBoost Classifier	0.854	0.917	0.729	0.600	0.657

Лучшей выбрана **Gradient Boosting Classifier**.

5.3. Настройка гиперпараметров

С помощью `tune_model(optimize='F1', n_iter=20)` выполнен поиск оптимальных гиперпараметров. По результатам PyCaret сообщил, что исходная модель оказалась лучше настроенной, поэтому финальной осталась базовая модель.

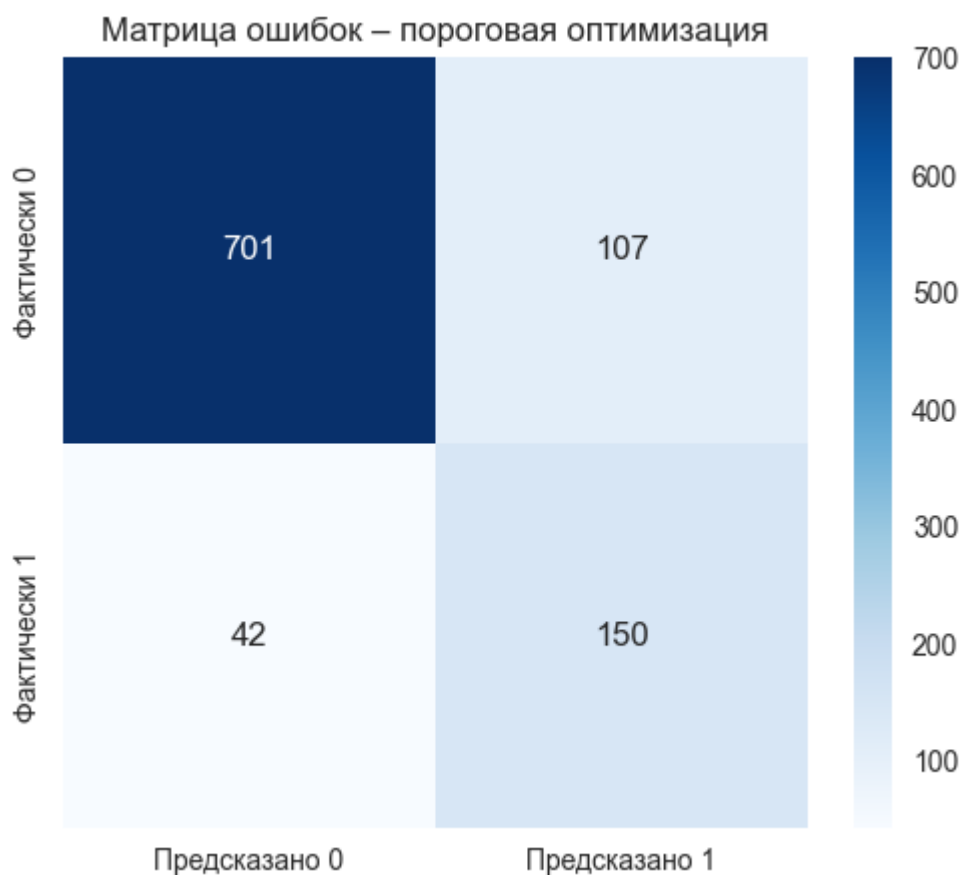
6. Оценка модели на тестовой выборке

Финальная модель применена к отложенному тестовому набору (1000 объектов). Дополнительно был выполнен поиск порога классификации по PR-кривой с целевыми значениями Precision ≥ 0.65 и Recall ≥ 0.70 . Оптимальный порог не был найден, поэтому использован стандартный порог 0.5.

Метрики классификации:

Класс	Precision	Recall	F1-score	Support
Нездоров (0)	0.94	0.87	0.90	808
Здоров (1)	0.58	0.78	0.67	192
Accuracy			0.85	1000
Macro avg	0.76	0.82	0.79	1000
Weighted avg	0.87	0.85	0.86	1000

Матрица ошибок:

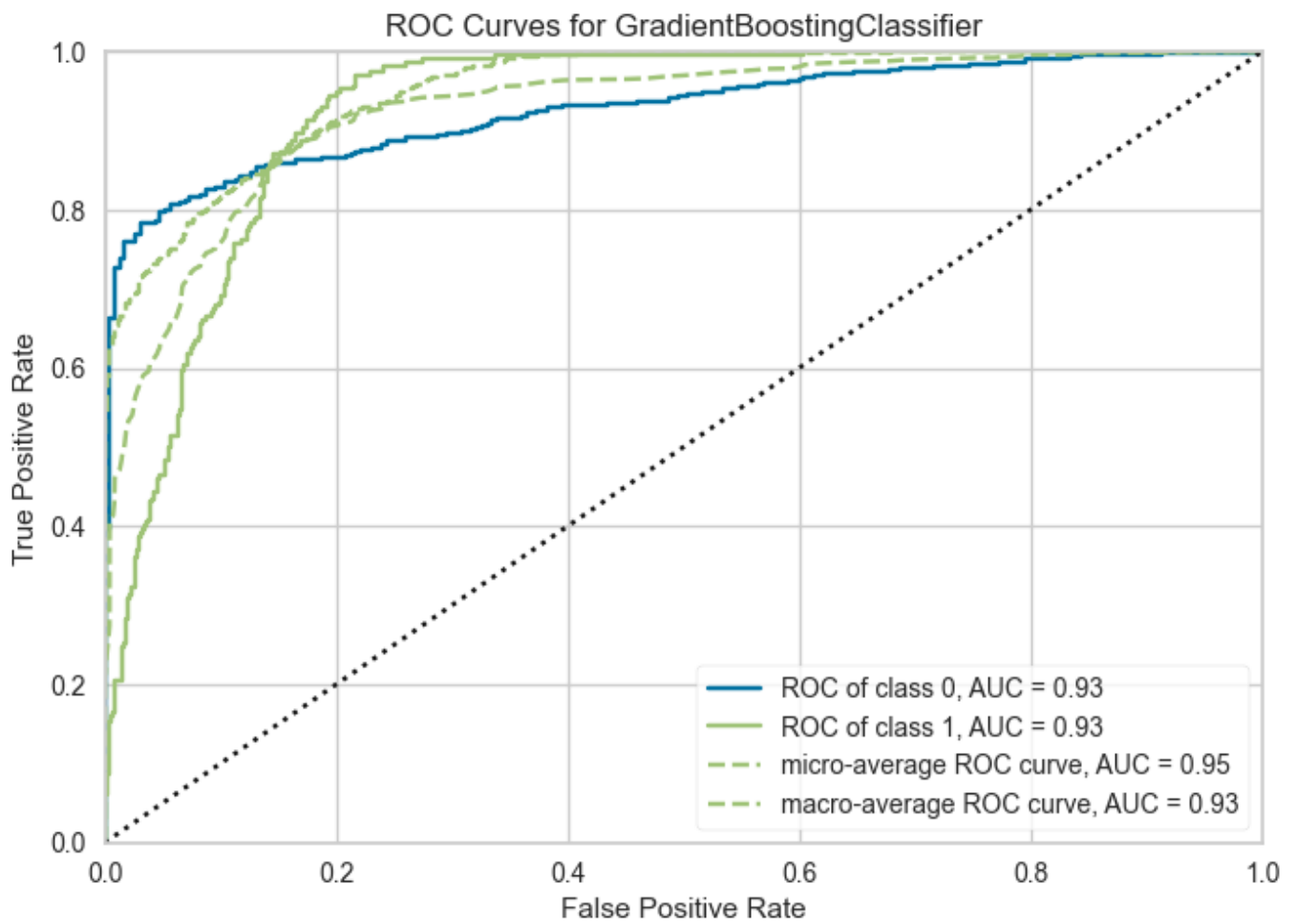


	Предсказано 0	Предсказано 1
Фактически 0	701	107
Фактически 1	42	150

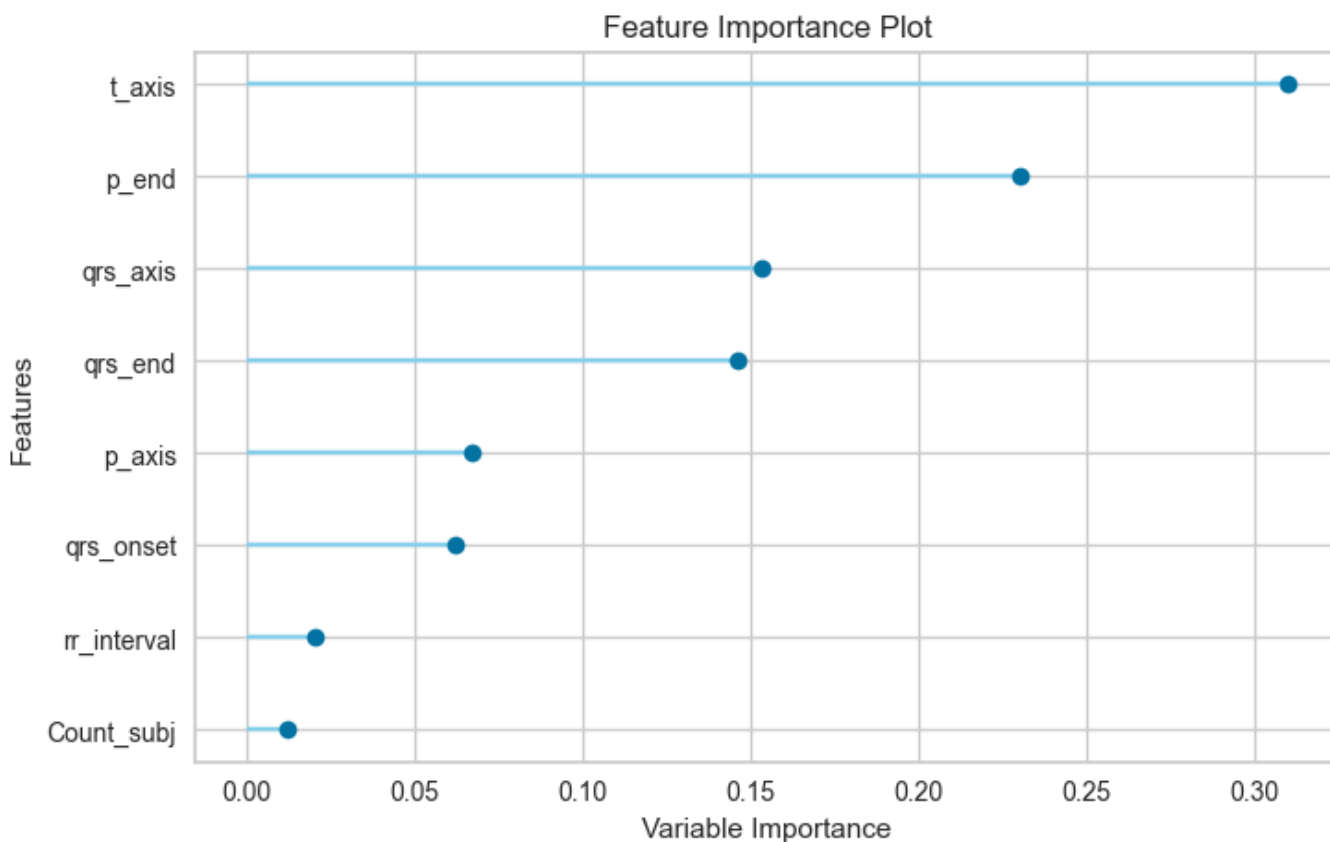
Интерпретация:

- Из 808 нездоровых пациентов 699 верно классифицированы, 109 ошибочно отнесены к здоровым.
- Из 192 здоровых пациентов 150 распознаны правильно, 42 пропущены.

Дополнительные визуализации:



$AUC = 0.918$ – модель обладает хорошей разделяющей способностью.



Наибольший вклад вносят `rr_interval` , `qrs_onset` , `p_end` и `t_axis` .

7. Обсуждение результатов

1. Влияние дисбаланса классов

Исходное соотношение классов 4.2:1 было смягчено SMOTE с параметром 0.4, что позволило модели не игнорировать миноритарный класс. В результате достигнут баланс между Precision и Recall для класса «Здоров».

2. Медицинская интерпретация

Модель с точностью 58% для положительного класса и полнотой 78% может быть использована в качестве скринингового инструмента: большинство больных будет выявлено, но при этом около половины предсказаний «здоров» окажутся ложными. Для ресурсоёмких методов диагностики такой уровень точности может потребовать доработки.

3. Пути улучшения

- Дальнейшая настройка порога классификации.
- Использование моделей с встроенной регуляризацией дисбаланса (CatBoost, LightGBM).
- Обработка выбросов, присутствующих в исходных данных.

8. Выводы

1. Числовые параметры ЭКГ позволяют построить модель бинарной классификации состояния здоровья с удовлетворительным качеством.
2. PyCaret как AutoML-инструмент обеспечил быстрое сравнение алгоритмов и настройку гиперпараметров.
3. Лучшая модель – **Gradient Boosting Classifier** – на тестовой выборке продемонстрировала:
 - **F1-мера = 0.67** для класса «Здоров»;
 - **Accuracy = 0.85**;
 - **AUC = 0.918**.
4. Основной проблемой остаётся низкая точность для класса здоровых пациентов (0.58), вызванная дисбалансом данных. Для её повышения требуется дополнительная настройка порога или использование других методов борьбы с несбалансированностью.
5. Все поставленные задачи выполнены: загружены и подготовлены данные, проведён разведочный анализ (PCA, t-SNE), обоснован выбор AutoML, построена и оценена модель, рассчитаны матрица ошибок и F1-метрика.